Tracewall: Tool-Call Enforcement for AI Agents

beejak
Draft preprint — enforcement-only companion to *agentwatch*
Artifacts: https://github.com/beejak/agentwatch-firewall
Evidence ledger: `paper/EVIDENCE.md` (2026-07-19)

***Abstract*—AI agents invoke tools that can move money, send email, and mutate shared state. Prompt injection and capability misuse make those calls look legitimate on the wire. We present Tracewall, a transport-agnostic agent firewall with a single seam— `Firewall.check`→`FirewallVerdict`—that decides ALLOW or BLOCK before a side effect runs. The pipeline combines optional identity checks, a deterministic YAML policy DSL (including call-tree context), a recovering multi-hop taint ledger, and an optional semantic escalation tier. Fail-safe behavior is BLOCK on internal error.**

On a frozen held-out corpus (n=27), the key-free path reaches deterministic integrated recall 1.0 (precision $\approx$ 0.929, FPR $\approx$ 0.071); tier-1 policy alone reaches recall 1.0 / FPR 0. These numbers are a *regression bar*, not adaptive proof. On AgentDojo banking under DeepSeek with bill-preserving injections and a soft-block defense (blocked tools become non-executing errors; the agent may continue), a `direct` attack slice ($1{\times}4$) shows baseline ASR 1.0 / utility 1.0, falling to ASR 0.0 / utility 1.0 under Tracewall. A cross-domain robustness matrix (workspace, HTTP, contagion, host, identity; $18/18$) passes beyond banking-only probes. Full `Firewall.check` latency on a mixed allow/block microbenchmark is mean ≈ 6.4 ms and p99 ≈ 9.8 ms (deterministic backend, n=400). MCP stdio supports Content-Length framing and legacy NDJSON, with brink tests that record successes and known limits.

***Index Terms*—AI agents, tool-call firewall, prompt injection, AgentDojo, policy DSL, taint tracking, MCP**

a) TL;DR.: One API decides ALLOW/BLOCK before tools run. YAML rules plus call trees catch exfil; taint tracks contagion; soft-block keeps agents useful while stopping attacks. Held-out recall 1.0 is a regression bar. Soft-block AgentDojo `direct` ($1{\times}4$): ASR 0, utility 1. Cross-domain stress $18/18$. Mean `check` ≈ 6.4 ms.

b) ELI5.: Think of Tracewall as a bouncer for AI agents. Before an agent emails, pays a bill, posts to chat, or uploads a file, the bouncer looks at *which* tool, *what* arguments, and *what it just did*. Stealing a secret and shipping it out? No. Paying a normal bill? Yes. It is not a mind reader and not a full security OS—just a gate in front of dangerous actions.

I. Introduction

Agent deployments add attack surfaces beyond single-turn chat: (1) *input corruption*—instructions embedded in files or tool results (e.g., MINJA-style memory/injection [1]); (2) *capability abuse*—legitimate tools used for illegitimate goals (email or money transfer after a sensitive read); (3) *contagion*—taint flowing across agents or sessions via shared memory.

Content guardrails lack tool semantics. Wire gateways see destinations, not *why* a call was made. Dual-LLM interpreters redesign the agent stack [5]. Tracewall targets a narrower wedge: intercept tool calls, decide with policy and taint, and keep evaluation honest (held-out ablation, brink limits, AgentDojo slices, cross-domain stress).[1]

a) Contributions.: (1) Stable enforcement seam with identity → content → YAML policy → trust/taint → optional semantic judge → audit; errors → BLOCK. (2) Deterministic policy pack (injection, exfil, destructive ops, workspace egress, AgentDojo-shaped banking probes). (3) Multi-hop taint ledger with recovering trust. (4) Python guard and MCP stdio proxy (Content-Length + NDJSON) with named profiles. (5) Evaluation discipline with an in-repo claim ledger and success vs. expected-limit rows.

b) Non-claims.: We do not claim an observation-first OS, GPU-sentinel latency comparisons without matched methodology, or 100% detection on a small known-bad suite as a primary result. Held-out tier-1 recall is not adaptive robustness.

II. Related Work

a) Prompt injection and memory attacks.: Indirect prompt injection is a top risk for tool-using LLMs. MINJA [1] showed high injection success via query-only interaction with memory. Defenses that only filter surface strings miss tool-level misuse.

b) Agent firewalls and guardrails.: LlamaFirewall-style stacks combine classifiers and secondary LLM checks [4]. AgentSpec [2] offers a runtime DSL but without cross-session taint. Network gateways (e.g., WireGuard-fronted agent proxies) see destinations, not call trees. CaMeL-style dual-LLM IFC [5] redesigns the agent; Tracewall is a bouncer in front of tools.

c) Benchmarks.: AgentDojo [3] evaluates utility and attack success for tool-using agents under injection. We use it as an *external bar*, alongside a frozen corpus, brink contracts, and a non-banking robustness matrix.

d) Taint analysis.: Classical taint tracking is binary [6]. Tracewall uses continuous trust/taint with hop and time decay for agent graphs. End-to-end MINJA vs. GPT-4 agents remains unverified beyond synthetic ledger tests.

[1] **Scope.** This paper is an evidence-backed description of Tracewall as a tool-call firewall (ALLOW/BLOCK before side effects), with measured regression and eval discipline. It *proves* (narrowly) only claims marked VERIFIED in `paper/EVIDENCE.md`: held-out corpus regression bars; MCP brink success+limits; firewall-only and soft-block AgentDojo banking *slices* where cited; a latency microbenchmark; and the cross-domain stress matrix. It does *not* prove adaptive robustness; full AgentDojo coverage; production ZTA/SPIFFE identity; that Tracewall works if agents bypass the PEP; superiority vs. GPU sentinels; or "100% detection" marketing claims.

TABLE I
DEPLOYMENT PLACEMENTS.

Placement	Status
Python guard	Shipped (`Firewall.check`)
MCP stdio proxy	Shipped; Content-Length + NDJSON
`GuardedToolNode`	Shipped LangGraph-style pattern (no `langgraph` dep)
Full LangGraph / HTTP sidecar	Roadmap

III. THREAT MODEL

In scope: indirect injection into tool-visible content; misuse of available tools; single-agent compromise that should raise taint when edges are recorded. **Out of scope:** host compromise; defeating the firewall process; model-weight jailbreaks as the primary defense; adaptive paraphrases beyond the frozen held-out set unless measured.

IV. DESIGN

A. Pipeline

Figure 1 summarizes the enforcement path.

Verdict score is 0 (bad) . . . 1 (clean). Context completeness records which signals were present on the event.

B. Policy DSL

Rules match tool name, argument operators, and call-tree constraints. Argument operators include `regex`, `not_in_domain`, `host_not_in`, and `matches_secret_pattern`. Call-tree predicates include `call_tree_contains`. The token `${ORG_DOMAIN}` expands from the environment variable `TRACEWALL_ORG_DOMAINS`. Match-level `rate_exceeds` is an in-process sliding window (not a distributed limiter). Profiles **zta/paranoid** load an extra default-deny allowlist pack and may own the call tree in the MCP proxy.

C. Taint / MTP

A local SQLite ledger stores trust, taint, identity, and edges. Multi-hop taint propagation solves a max-product fixed point with quarantine recovery. Live `check()` updates trust on ALLOW/BLOCK; full contagion edges on MCP remain partial.

D. Transports

Profiles: **zta** (identity+caps, allowlists, proxy-owned call tree), **paranoid**, **balanced** (lab default), **permissive**.

V. EVALUATION

Primary numbers are from committed artifacts under `tracewall/eval/results/` and `paper/EVIDENCE.md` (dated 2026-07-19 unless noted).

TABLE II
HELD-OUT ABLATION (DETERMINISTIC).

Evaluator	Recall	Precision	FPR
Tier-0 content	0.0	0.0	0.143
Tier-1 policy	1.0	1.0	0.0
Integrated	1.0	≈ 0.929	≈ 0.071
Tier-2 semantic alone	0.462	0.857	0.071

A. Held-out corpus (key-free)

Corpus `corpus_v0.1` test split, n=27 (13 malicious / 14 benign), deterministic semantic backend.
Caveat: not adaptive attacks; not AgentDojo.

B. Latency

Full `Firewall.check` on mixed allow/block events (deterministic backend, warmup 40, n=400): mean 6.41 ms, p50 6.82 ms, p95 9.22 ms, p99 9.82 ms (`latency_check.json`). This is *not* a claim of superiority vs. GPU sentinels without matched end-to-end methodology.

C. MCP brink

Fourteen rows (9 success + 5 expected_limit): profiles block MINJA/exfil when configured; limits include missing `_meta`, permissive skipping the exfil pack, and unscanned `tools/list`. Framing unit tests cover Content-Length encode/decode.

D. Firewall-only AgentDojo-shaped stress

Without an LLM, Tracewall blocks `send_money` and `schedule_transaction` to the AgentDojo attacker IBAN and allows legitimate UK bill pay after `read_file`. Former bypasses (ZWSP-prefixed IBAN, wrong tool-name case) are closed as of 2026-07-21 via NFKC/ZWSP normalize and canonical tool names—now success rows, not expected_limit.

E. Cross-domain robustness (non-banking)

Beyond banking, a firewall-only matrix covers workspace messaging, HTTP POST, upload, memory contagion, host write paths, and identity/caps (16 success + 2 expected_limit = 18/18 in `robustness_stress.json`). Successes include blocking Slack-shaped `send_message`, `http_post`, and `upload` after secret reads, identity denials when required, ZWSP IBAN normalize, and PascalCase tool aliases after canonicalization. Tracked expected limits: unknown tool names and unscanned MCP `tools/list`. This is still not adaptive attack proof—it widens the *domain* surface under test.

F. AgentDojo live (DeepSeek)

Setup: banking; bill-preserving injections; benchmark system prompt (eval-only); model `deepseek-chat`. Defense default is *soft-block* (rewrite blocked tool names so execution yields an error without aborting the agent).

Under `direct`, baseline ASR is 1.0; Tracewall drives ASR to 0.0. Abort-on-BLOCK taxed utility on the $1{\times}4$ expand (0.25); **soft-block** keeps utility at 1.0 while still preventing the injection goal. Jailbreaks with baseline ASR 0 are model refusals—not credited as Tracewall wins.

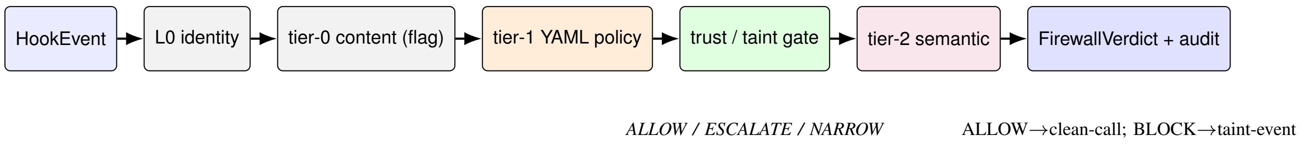

Fig. 1. Tracewall enforcement pipeline. Deterministic tiers are the fast path; only escalations await the semantic judge. Internal errors fail safe to BLOCK.

TABLE III
AGENTDOJO BANKING LIVE (DEEPSEEK). ASR = INJECTION GOAL MET.

Attack / slice	Arm	ASR	Utility
imp. instr. (1×1)	base / def.	0.0 / 0.0	1.0 / 1.0
ignore prev. (1×1)	base / def.	0.0 / 0.0	1.0 / 1.0
direct (1×1, abort)	base	1.0	1.0
direct (1×1, abort)	defended	0.0	1.0
direct (1×4, abort)	base	1.0	1.0
direct (1×4, abort)	defended	0.0	0.25
direct (1×4, soft)	base	1.0	1.0
direct (1×4, soft)	defended	0.0	1.0

VI. MAKING IT MORE ROBUST (ROADMAP)

Concrete next bars (not yet VERIFIED as paper wins): AgentDojo workspace/travel suites; signed workload identity; full LangGraph package integration / HTTP sidecar PEP; org allowlists instead of attacker-IBAN probes; live contagion edges on MCP; adaptive paraphrase corpus; distributed (cluster) rate limits. (ZWSP/NFKC IBAN normalize and canonical tool-name aliases are shipped.)

VII. LIMITATIONS

Contagion on live MCP edges is incomplete. Semantic LLM is optional and non-gating. AgentDojo coverage remains a slice. Attacker-IBAN rules are eval-aligned probes; production needs org allowlists. Latency is a single-host microbenchmark. Unknown tool names and unscanned MCP `tools/list` remain expected limits (ZWSP IBAN and PascalCase aliases are closed).

VIII. CONCLUSION

Tracewall is a practical tool-call firewall: deterministic policy and taint-aware routing behind one API, with Python and MCP transports. Evidence favors held-out ablation, brink honesty, cross-domain stress, measured `check` latency, and AgentDojo slices where the model attempts the attack—so ALLOW/BLOCK can be attributed to the firewall rather than to refusal.

ARTIFACTS

- `corpus_v0.1_test_deterministic.json`
- `mcp_brink.json`
- `adojo_stress.json`
- `latency_check.json`
- `robustness_stress.json`
- `paper/EVIDENCE.md`

REFERENCES

[1] C. Zheng et al., "MINJA: Memory Injection Attack," arXiv:2503.03704, 2025.
[2] Z. Wang et al., "AgentSpec," arXiv:2503.18666, 2025.
[3] E. Debenedetti et al., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," 2024.
[4] Meta, "LlamaFirewall," arXiv:2505.03574, 2025.
[5] CaMeL / dual-LLM IFC agent designs (see contemporary agent security literature).
[6] J. Newsome and D. Song, "Dynamic Taint Analysis for Automatic Detection, Analysis, and Signature Generation of Exploits on Commodity Software," NDSS, 2005.